Classification and Regression Trees (CART)

Growing, regularizing and understanding decision trees — and why we need Random Forests

Anca Ralescu

Machine Learning Course

Outline

1. Decision Trees: the Basic Idea
2. The CART Training Algorithm
3. Classification Trees on the Iris Data
4. Regularization
5. Regression Trees
6. Limitations of Decision Trees
7. From Trees to Random Forests
8. Summary
9. APPENDIX

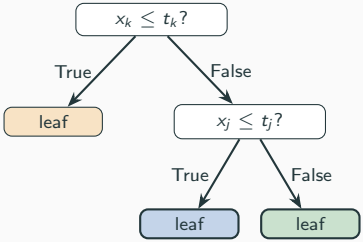
Decision Trees: the Basic Idea

What is a decision tree?

- A hierarchy of simple questions about the features:

  "is $x_k \leq t_k$?"

- Each **internal node** tests one feature k against a threshold t_k.
- Each **leaf** stores a prediction:
  - a class (and class proportions) for *classification*;
  - a real value for *regression*.
- Prediction = follow the path from root to a leaf.
- A white-box model: decisions are easy to read and interpret.



Root at depth 0; each split adds one level.

Notation

- Training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$, with $\mathbf{x}^{(i)} \in \mathbb{R}^n$.
- m = number of training instances, n = number of features.
- Classification: $y^{(i)} \in \{1, \ldots, K\}$; Regression: $y^{(i)} \in \mathbb{R}$.
- A split is a pair (k, t_k): feature index $k \in \{1, \ldots, n\}$ and threshold $t_k \in \mathbb{R}$.

$$\text{left} = \{i : x_k^{(i)} \leq t_k\}, \qquad \text{right} = \{i : x_k^{(i)} > t_k\}.$$

- $m_{\text{left}}, m_{\text{right}}$ = number of instances sent left / right, $m_{\text{left}} + m_{\text{right}} = m$.
- For node i: $p_{i,k}$ = fraction of the training instances in node i that belong to class k.

Running example

Iris data, using only the two features *petal length* and *petal width*; $m = 150$, $K = 3$ classes (Setosa, Versicolor, Virginica), 50 of each.

The CART Training Algorithm

CART: growing a tree

Simple idea

1. **Split** the training set in two subsets using a single feature k and a threshold t_k (e.g. *petal length* ≤ 2.45 cm).
   - To choose k and t_k: search for the pair (k, t_k) that produces the purest subsets, weighted by their size.
2. **Split the subsets** by the same logic (recursion).
3. **Stop** once the tree reaches a maximum depth, or if no split which reduces impurity can be found.

- CART always produces **binary trees**: every internal node has exactly two children.
- Candidate thresholds: midpoints between consecutive sorted values of feature k in the node.

CART cost function for classification

For a candidate split (k, t_k), CART minimizes

$$\boxed{J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}}$$

where

- $G_{\text{left/right}}$ measures the impurity of the left/right subset;
- $m_{\text{left/right}}$ is the number of instances in the left/right subset;
- m is the number of instances in the node being split.

The best split is

$$(k^\star, t_k^\star) = \arg \min_{k \in \{1..n\},\ t_k} J(k, t_k).$$

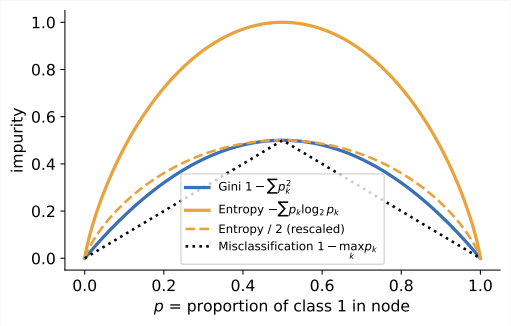
Equivalently, CART maximizes the **impurity decrease**

$$\Delta G = G_{\text{parent}} - J(k, t_k) \geq 0.$$

Gini impurity index

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

- $p_{i,k}$: ratio of class-k instances among the training instances in node i.
- $G_i = 0 \iff$ node is pure (all instances in one class).
- Maximum $1 - 1/K$ when classes are equally mixed.
- Equivalent form: $G_i = \sum_k p_{i,k}(1 - p_{i,k})$ — probability of mislabelling a random instance labelled at random from the node's class distribution.



Impurity measures for a two-class node as a function of the class-1 proportion p.

Entropy as an alternative impurity measure

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{K} p_{i,k} \log_2 p_{i,k}$$

- Originates from information theory (average information content of a message).
- $H_i = 0$ for a pure node, maximum $\log_2 K$ for a uniform mixture.
- Use H in place of G in $J(k, t_k)$; the decrease $H_{\text{parent}} - J$ is the information gain.

Example — the depth-2 left node of the Iris tree: $\mathbf{p} = (0,\ 49/54,\ 5/54)$

$$G = 1 - \left(\tfrac{49}{54}\right)^2 - \left(\tfrac{5}{54}\right)^2 \approx 0.168, \qquad H = -\tfrac{49}{54}\log_2 \tfrac{49}{54} - \tfrac{5}{54}\log_2 \tfrac{5}{54} \approx 0.445.$$

Gini impurity versus entropy

- The choice makes **no big difference** with respect to the final result: the trees obtained are quite similar.
- Gini impurity is **slightly faster to compute** (no logarithms) $\Longrightarrow$ it is a good default.
- When the trees differ:
  - Gini tends to isolate the most frequent class in its own branch of the tree;
  - Entropy tends to produce slightly more balanced trees.

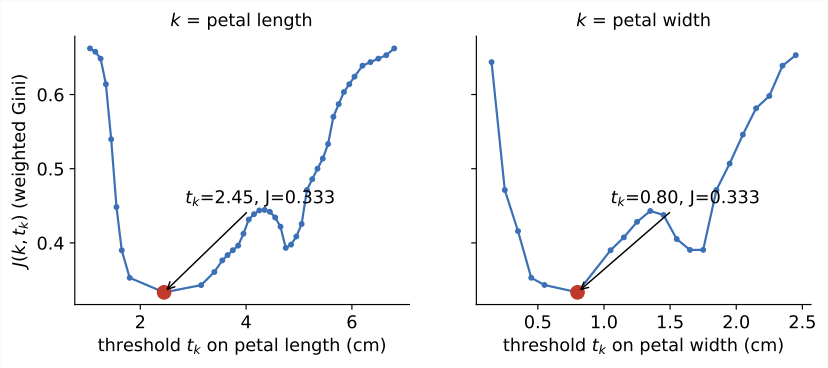
	pure node	2 classes, 50/50	K classes, uniform
Gini G	0	0.5	$1 - 1/K$
Entropy H	0	1	$\log_2 K$
Misclassification $1 - \max_k p_k$	0	0.5	$1 - 1/K$

CART pseudocode

function GrowTree($\mathcal{D}$, depth)
  if depth $=$ `max_depth` **or** $|\mathcal{D}| <$ `min_samples_split` **or** $\mathcal{D}$ is pure **then**
    return Leaf($\mathcal{D}$) ▷ majority class / class proportions / mean
  end if
  $(k^\star, t^\star) \leftarrow \arg\min_{k,t} \; J(k,t) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$
  if no split decreases impurity **then**
    return Leaf($\mathcal{D}$)
  end if
  $\mathcal{D}_L \leftarrow \{(\mathbf{x}, y) \in \mathcal{D} : x_{k^\star} \le t^\star\}, \quad \mathcal{D}_R \leftarrow \mathcal{D} \setminus \mathcal{D}_L$
  $T_L \leftarrow$ GrowTree($\mathcal{D}_L$, depth+1); $\quad T_R \leftarrow$ GrowTree($\mathcal{D}_R$, depth+1)
  return Node($k^\star, t^\star, T_L, T_R$)
end function

- Inner search: for each feature, sort the values once and sweep the candidate thresholds, updating the class counts incrementally.

Finding the best split: an example



Root of the Iris tree: **every threshold** on **each feature** is scanned and $J(k, t_k)$ evaluated. Both features reach the same minimum $J = \frac{50}{150} \cdot 0 + \frac{100}{150} \cdot 0.5 = 0.333$ (a tie: they separate Setosa perfectly). The tie is broken arbitrarily (random feature order).

CART is a greedy algorithm

- It greedily searches for an optimum split at the **top level**, then repeats the process at each subsequent level.
- It does not check whether or not the split will lead to the lowest possible impurity several levels down.
- It often produces a **reasonably good** solution, but it is **not guaranteed** to be optimal.

Finding the optimal tree is hard

Finding the optimal tree is an **NP-complete** problem: its time complexity is

$$O(e^m),$$

which is intractable even for small training sets. Hence we accept a "reasonably good" greedy solution.

Computational complexity

Prediction

- Traverse from the root to a leaf.
- A (roughly balanced) tree has depth $\approx \log_2(m)$.
- Each node checks *one* feature value.

$$\text{Prediction cost} = O\big(\log_2(m)\big)$$

independent of the number of features $\Rightarrow$ very fast.

Training

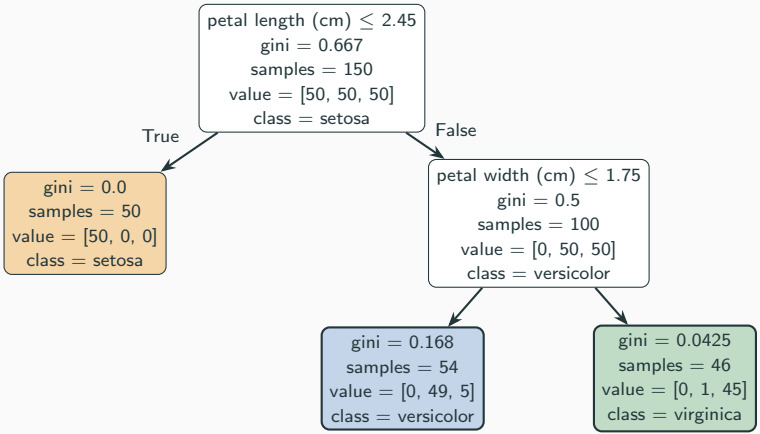
- At every node, compare all features on all samples in that node.
- Summed over the $\approx \log_2 m$ levels:

$$\text{Training cost} = O\big(n \times m \log(m)\big)$$

where n is the number of features (or the *maximum* number of features considered, `max_features`).

Classification Trees on the Iris Data

A classification tree for Iris (Gini, `max_depth`=2)



samples: instances reaching the node; **value**: instances per class; **gini**: impurity; **class**: majority class.

Checking the numbers

- Root: $G_0 = 1 - 3\left(\frac{50}{150}\right)^2 = 1 - \frac{1}{3} = 0.667$.
- Depth-1 right node: $G = 1 - \left(\frac{50}{100}\right)^2 - \left(\frac{50}{100}\right)^2 = 0.5$.
- Depth-2 leaves:

$$G_{\mathrm{L}} = 1 - \left(\tfrac{49}{54}\right)^2 - \left(\tfrac{5}{54}\right)^2 \approx 0.168, \qquad G_{\mathrm{R}} = 1 - \left(\tfrac{1}{46}\right)^2 - \left(\tfrac{45}{46}\right)^2 \approx 0.0425.$$

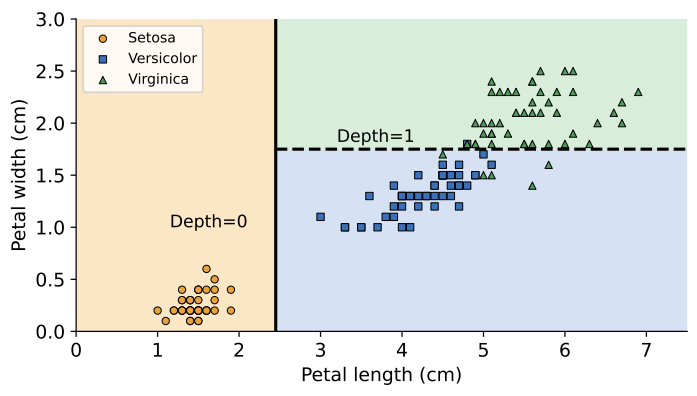
Costs of the two chosen splits

$$J_{\text{root}} = \tfrac{50}{150}(0) + \tfrac{100}{150}(0.5) = 0.333 \qquad \Rightarrow \Delta G = 0.667 - 0.333 = 0.333$$

$$J_{\text{depth 1}} = \tfrac{54}{100}(0.168) + \tfrac{46}{100}(0.0425) \approx 0.110 \qquad \Rightarrow \Delta G = 0.5 - 0.110 = 0.390$$

Weighted impurity after both splits: $\frac{54}{150}(0.168) + \frac{46}{150}(0.0425) \approx 0.073$.

Decision boundaries



Depth-0 split: vertical line *petal length* $= 2.45$ (left region is pure Setosa, cannot be split further). Depth-1 split: dashed line *petal width* $= 1.75$. With `max_depth`$= 2$ the tree stops here. Boundaries are always axis-parallel.

Estimating class probabilities

A decision tree can estimate the probability that an instance $\mathbf{x}$ belongs to class k:

1. Traverse the tree to find the leaf node $\ell(\mathbf{x})$ for instance $\mathbf{x}$;
2. Return the ratio of training instances of class k in this node:

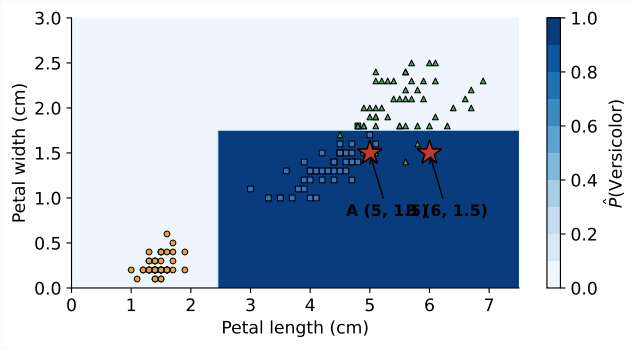
$$\widehat{P}(y = k \mid \mathbf{x}) = p_{\ell(\mathbf{x}),k} = \frac{\#\{\, i \in \ell(\mathbf{x}) : y^{(i)} = k \,\}}{m_{\ell(\mathbf{x})}}.$$

The predicted class is $\widehat{y} = \arg\max_k \widehat{P}(y = k \mid \mathbf{x})$.

Example: petals 5 cm long and 1.5 cm wide → depth-2 left leaf (54 samples)

$$P(\text{Setosa}) = \tfrac{0}{54} = 0, \qquad P(\text{Versicolor}) = \tfrac{49}{54} = 0.907, \qquad P(\text{Virginica}) = \tfrac{5}{54} = 0.093.$$

Probabilities are piecewise constant



Very important note

The estimated probabilities are **identical anywhere** in the bottom-right rectangle.

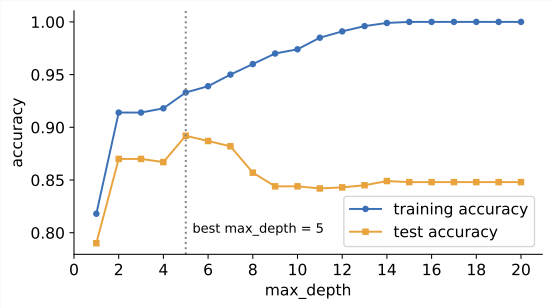
Point B (6 cm long, 1.5 cm wide) gets exactly the same $\widehat{P} = (0, 0.907, 0.093)$ as point A — even though it would most likely be an Iris *Virginica*.

Each leaf corresponds to one axis-aligned rectangle with a constant prediction.

Regularization

Why regularize?

- Decision trees make very few assumptions about the data (nonparametric model: the number of parameters is not fixed before training).
- Left unconstrained, the tree adapts itself to the training data, fitting it very closely ⇒ overfitting.
- To prevent this, we **restrict the freedom** of the tree during training: *regularization*.
- The quantities controlling these restrictions are called **hyperparameters**.



Moons data (noise 0.3): training accuracy keeps growing with depth; test accuracy peaks, then declines.

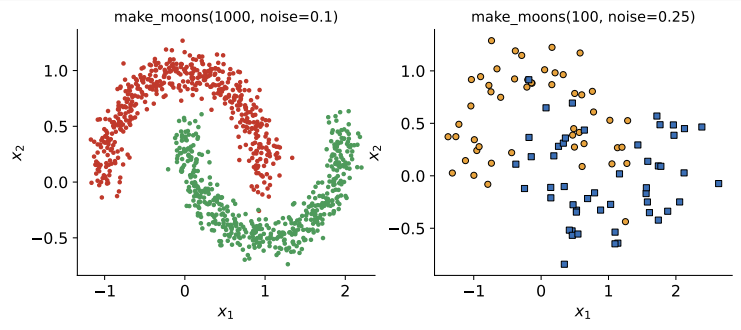
Regularization hyperparameters

Hyperparameter	Meaning
`max_depth`	maximum depth of the tree
`min_samples_split`	minimum number of examples a node must have before it can be split
`min_samples_leaf`	minimum number of samples a leaf node must have
`min_weight_fraction_leaf`	same as `min_samples_leaf`, but expressed as a fraction of the total number of weighted instances
`max_leaf_nodes`	maximum number of leaf nodes
`max_features`	maximum number of features evaluated for splitting at each node

Rule of thumb

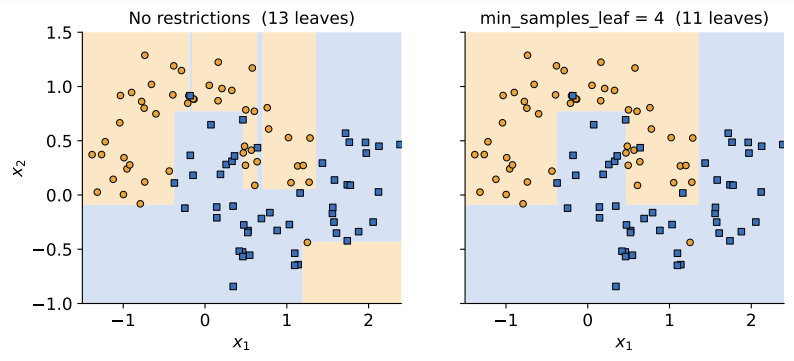
Increasing `min_*` hyperparameters or reducing `max_*` hyperparameters **regularizes** the model.

The moons dataset



Two interleaving half-circles: the classes are not linearly separable in the original 2-D space. The data could be mapped to a space of (much) higher dimension where the two classes become linearly separable — a decision tree instead carves the plane into rectangles.

Regularization using `min_samples_leaf`



- **Left**: default hyperparameters (no restrictions) — the tree isolates individual noisy points: overfitting.
- **Right**: `min_samples_leaf`$= 4$ — smoother, simpler boundary that will generalize better.

Pre-pruning versus post-pruning

Pre-pruning (early stopping)

Stop growing when a hyperparameter limit is hit: `max_depth`, `min_samples_leaf`, `max_leaf_nodes`, ...

This is what the hyperparameters above implement.

Post-pruning

Grow the full tree, then remove nodes whose children bring no *statistically significant* (or cost-justified) improvement.

Cost-complexity pruning (CART) minimizes

$$R_\alpha(T) = R(T) + \alpha\,|\widetilde{T}|,$$

$R(T)$: training error, $|\widetilde{T}|$: number of leaves, $\alpha \geq 0$: complexity penalty.

In both cases the regularization strength is chosen by cross-validation.

Regression Trees

Regression trees

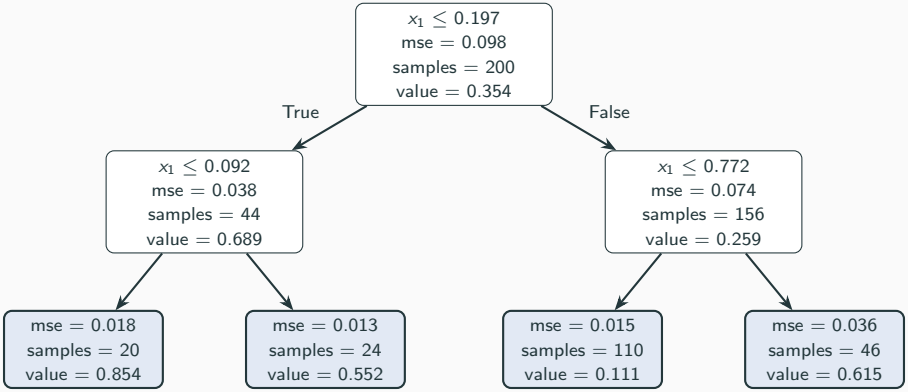
- Decision trees can perform **regression** tasks as well.
- The "label" y now has a continuous value: $y \in \mathbb{R}$.
- The tree looks like a classification tree, except that each node stores
  - a **value** $\widehat{y}_{\text{node}}$: the *average* target of the training instances in the node;
  - its **MSE**: mean squared error of the instances around that value.
- Prediction for $\mathbf{x}$: the value $\widehat{y}$ of the leaf reached by $\mathbf{x}$.

Running example

$m = 200$ points with $x_1 \sim \mathcal{U}(0, 1)$ and a noisy quadratic target

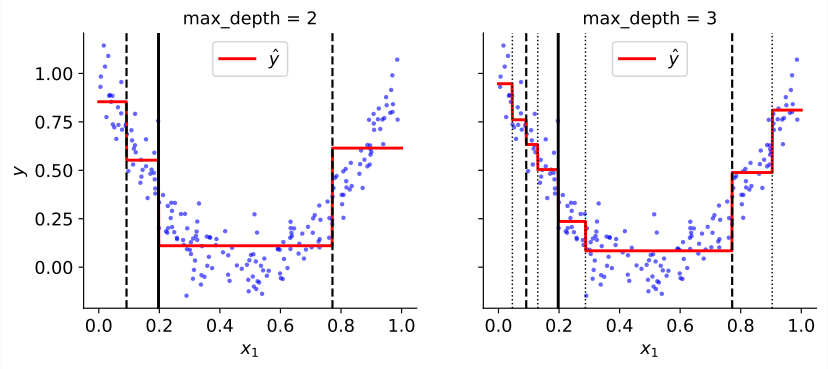
$$y = 4\,(x_1 - 0.5)^2 + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,\, 0.1^2).$$

A regression tree (max_depth=2)



E.g. a new instance with $x_1 = 0.6$: root $\rightarrow$ right ($0.6 > 0.197$) $\rightarrow$ left ($0.6 \leq 0.772$) $\Rightarrow \widehat{y} = 0.111$.

Regression tree predictions



Left: `max_depth`= 2 (4 leaves). Right: `max_depth`= 3 (8 leaves). Solid / dashed / dotted vertical lines: splits at depth 0 / 1 / 2. In each region the prediction is the mean of the training targets in that region.

CART cost function for regression

Replace impurity by the mean squared error:

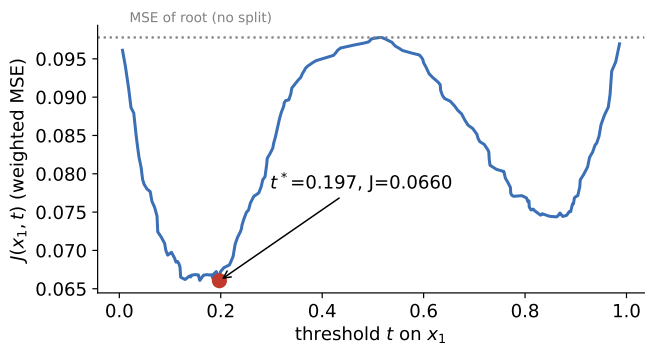
$$\boxed{J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}}$$

where, for a node with m_{node} instances,

$$\text{MSE}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} \left(\widehat{y}_{\text{node}} - y^{(i)} \right)^2, \qquad \widehat{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)}.$$

- $\widehat{y}_{\text{node}}$ is the constant that minimizes the squared error in the node.
- $m\, J(k, t_k)$ is the total within-node sum of squares after the split: CART chooses the split that reduces the variance the most.

Finding the best regression split

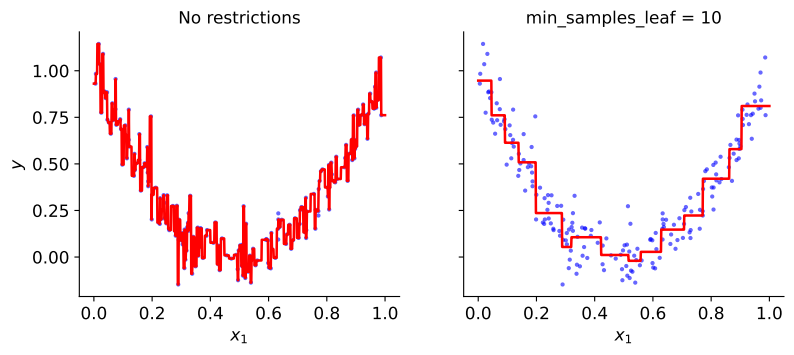


Weighted MSE $J(x_1, t)$ for every candidate threshold at the root.

The minimum is reached at $t^\star \approx 0.197$, the root split shown in the tree.

J drops from 0.098 (no split) to about 0.066.

Regularization of regression trees



- **Left**: no regularization — the tree has one leaf per training point and fits the noise.
- **Right**: `min_samples_leaf`$= 10$ — a much more reasonable model.
- Use the same restricted hyperparameters as for classification to avoid overfitting.

Limitations of Decision Trees

Strengths and weaknesses at a glance

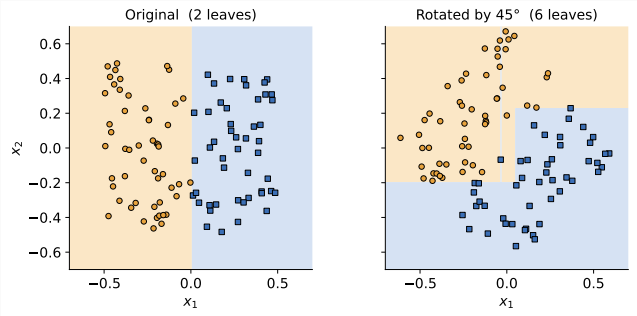
Strengths

- Easy to interpret (white box)
- Little data preparation: no feature scaling or centering needed
- Handles numerical and categorical features, multi-class problems
- Very fast prediction: $O(\log_2 m)$

Weaknesses

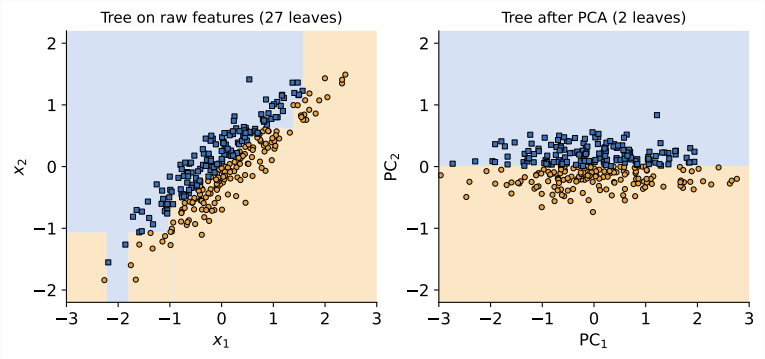
- Greedy: not globally optimal
- Prone to overfitting without regularization
- Orthogonal boundaries $\Rightarrow$ **sensitive to rotation**
- **Unstable**: very sensitive to small variations in the training data

Instability to rotation



- Decision trees have **orthogonal** decision boundaries (all splits are perpendicular to an axis).
- **Left**: a simple linearly separable data set — one split suffices.
- **Right**: after a 45° rotation, the tree produces an unnecessarily convoluted staircase boundary.
- Both fit the training set perfectly, but the right model will very likely not generalize well.

Remedy: Principal Component Analysis



PCA rotates the data onto the directions of largest variance:

$$\mathbf{z} = \mathbf{W}^\top(\mathbf{x} - \bar{\mathbf{x}}),$$

where the columns of $\mathbf{W}$ are the eigenvectors of the covariance matrix.

The informative direction becomes axis-aligned, so the tree needs far fewer splits and generalizes better.

Instability to details in the data

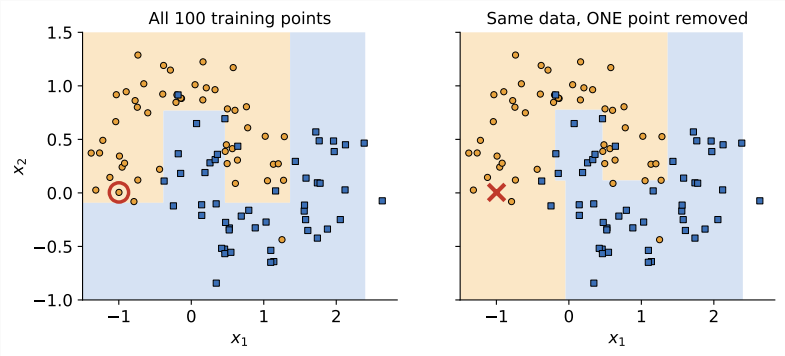
The main issue with decision trees is their extreme sensitivity to small variations in the training data.

Example (Iris)

Removing the widest Versicolor from the iris training set (the one with petals 4.8 cm long and 1.8 cm wide) and re-training a decision tree may result in a decision surface that looks very different from the original one.

- Ties between equally good splits (e.g. *petal length* ≤ 2.45 vs. *petal width* ≤ 0.8 at the Iris root) are broken at random: even the `random_state` can change the tree.
- Because splitting is greedy and hierarchical, a change near the root **propagates** to the whole subtree.
- Statistically: a single tree is a **high-variance** estimator.

Sensitivity to one training point



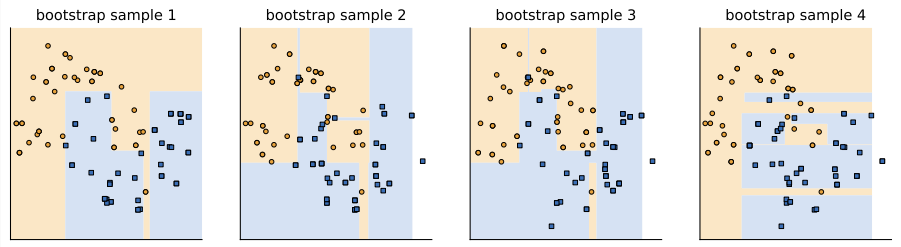
Moons data, `min_samples_leaf` $= 4$.

Removing the single circled point (right, marked $\times$) changes the predicted class over roughly 18% of the plane.

A small perturbation of the data leads to a visibly different model.

From Trees to Random Forests

Averaging away the variance



- Unregularized trees trained on four bootstrap samples of the *same* data: four quite different models.
- Each is **low bias, high variance**. Idea: average many of them.
- If B predictors each have variance σ^2 and pairwise correlation ρ, their average has variance

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x})\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

Bagging (bootstrap aggregating)

1. For $b = 1, \dots, B$: draw a **bootstrap sample** $\mathcal{D}_b$ of size m from $\mathcal{D}$ *with replacement*, and grow a tree $\hat{f}_b$ on it.
2. Aggregate:

$$\text{regression: } \hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(\mathbf{x}), \qquad \text{classification: } \hat{y}(\mathbf{x}) = \arg\max_k \sum_{b=1}^{B} \mathbb{1}\{\hat{f}_b(\mathbf{x}) = k\}.$$

(Soft voting averages the class probabilities $\hat{P}_b(y = k \mid \mathbf{x})$ instead.)

Out-of-bag (OOB) evaluation

Probability that a given instance is *not* drawn in a bootstrap sample:

$$\left(1 - \tfrac{1}{m}\right)^m \xrightarrow[m\to\infty]{} e^{-1} \approx 0.368.$$

Each tree ignores $\approx 37\%$ of the data; predicting each instance with the trees that never saw it gives a free validation estimate.

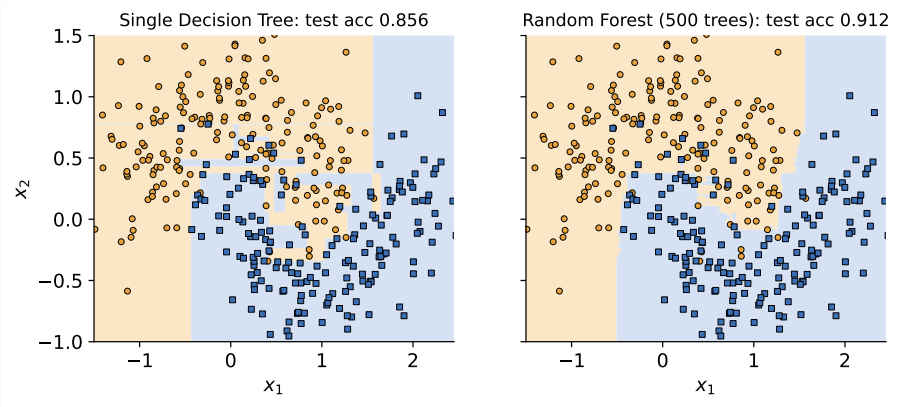
Random Forests

- Bagging alone leaves the trees **correlated** (they all pick the same strong features near the root) $\Rightarrow$ the term $\rho\sigma^2$ does not vanish.
- **Random Forest** = bagging of trees + random feature subspace at every split: only a random subset of `max_features` features is searched,

$$\texttt{max_features} \approx \sqrt{n}\ \text{(classification)}, \qquad \approx n/3 \text{ or } n\ \text{(regression)}.$$

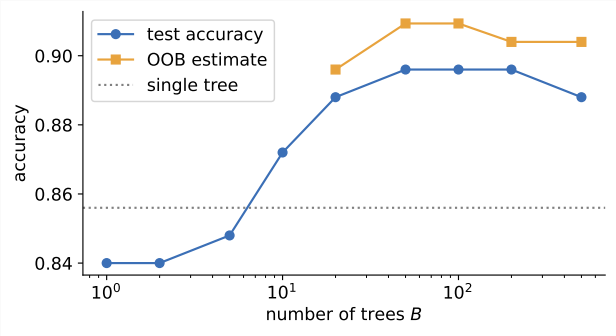
- This **decorrelates** the trees (lower ρ) at the price of a slightly higher bias per tree $\Rightarrow$ lower overall variance.
- **Extra-Trees** (extremely randomized trees) go further: thresholds are also drawn at random instead of optimized — even faster to train.

Random Forest versus a single tree



Moons data, 500 points, noise 0.3 (375 train / 125 test). The forest (500 trees, `max_leaf_nodes`= 16) has a much smoother decision boundary and a higher test accuracy (0.912 vs. 0.856 for a single unrestricted tree).

How many trees?



- Test accuracy improves quickly with the number of trees B, then stabilizes.
- Adding trees does **not** cause overfitting: it only reduces the variance term $\frac{1-\rho}{B}\sigma^2$.
- The OOB estimate tracks the test accuracy without a separate validation set.

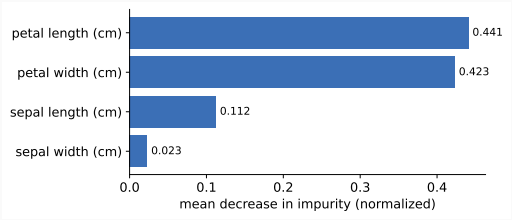
Feature importance

- Mean Decrease in Impurity (MDI) of feature j:

$$\mathrm{Imp}(j) = \frac{1}{B}\sum_{b=1}^{B}\sum_{\substack{t\in T_b \\ k(t)=j}} \frac{m_t}{m}\,\Delta G_t$$

  summed over all nodes t that split on j, weighted by the fraction of samples m_t/m reaching t; then normalized to sum to 1.

- Useful for **feature selection** and interpretation.
- On the full Iris data, petal measurements dominate.



Summary

Summary

- CART grows **binary trees** greedily by minimizing

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}},$$

  with G = Gini (or entropy) for classification, and MSE for regression.
- Greedy $\Rightarrow$ good but not optimal; the optimal tree is NP-complete, $O(e^m)$.
- Complexity: training $O(n\, m \log m)$, prediction $O(\log_2 m)$.
- Leaves give class proportions $\Rightarrow$ probabilities that are **piecewise constant**.
- Trees overfit: regularize with `max_*` / `min_*` hyperparameters or pruning.
- Limitations: axis-parallel boundaries (rotation sensitivity; PCA helps) and high variance (instability to data details).
- Averaging many decorrelated trees — **Random Forests** — reduces that variance.